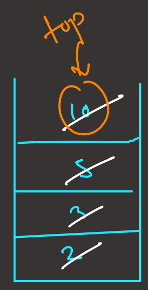
Stack

LIFO Data Structure

insert order → [2 3 5 10]



[10 5 3 2]

Operation

1. push() ← last in
2. pop() → top out
3. peak() → top point

implementation

1. Array → fixed size → problem
2. ArrayList → dynamic → [ok-ok]
3. Linked List. → Dynamic → Control on developer

AL

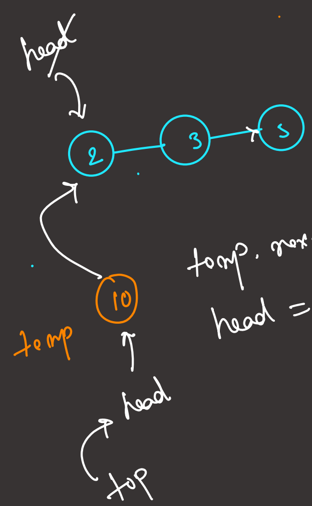
list → | 2 | 3 | 5 | 10 |

push(15) O(1)

list.add();

pop : list.remove() O(1)

peak : list.get(size()-1); O(1)



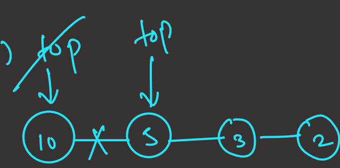
O(1)

temp.next = head

head = temp

push(10)

pop()



O(1)

JCF

```
Stack<Integer> s = new Stack<>();
```

Question 1

push at the bottom of the stack.

push → 4

3
2
1

4 →

3
2
1
4

O(n)

Space = O(n)

3
2
1
4

~~1~~
~~2~~
~~3~~

Way (1)

using recursion

```
fun (S, data) {
    if (S.isempty ()) {
        S. push () ; return ;
    }
    int (x) = pop ()   [ Stack → top ]
    fun ()
    push (x) ;
}
```

S. push (data)

funct S = empty()

funct, x = 1

funct x = 2

x = 3
func (S, data)

Stack is created by recursion call

T.C = O(n)

~~3~~
~~2~~
~~1~~

3
2
1
4

Question 2

Reverse a String using Stack.

Str = "abc"

```
for (int i = 0 to str.length) {
    char c = str.charAt(i)
    s.push (c) ;
}
```

~~c~~
~~b~~
~~a~~

"c b a"

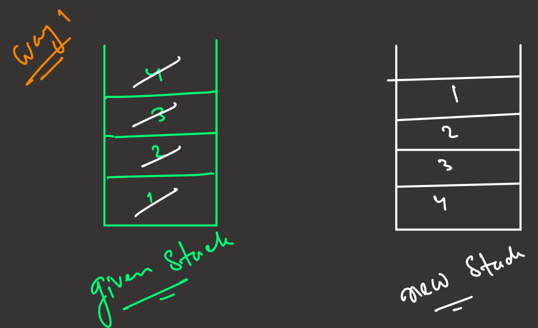
Stack <Character> S = new Stack <>() ;

```
while (!s.isEmpty ()) {
    Sout ( s.pop () ) ;
}
```

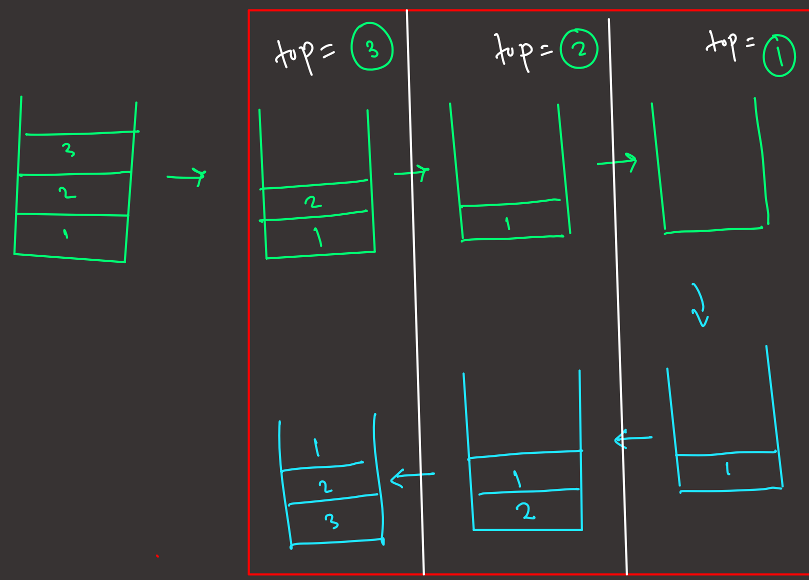
Question 3

reverse a Stack

Way 1



Way 2




```
top = s.pop();
reverse() // recursive call
pushAtBottom(top);
```

Question 4

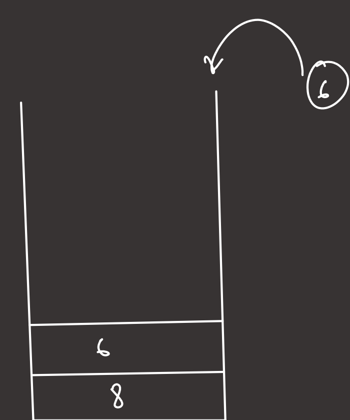
Next greater Element [from right]

Next greater element of some element x in an array is the first greater element that is to the right of the x in the same array.

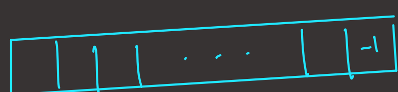
arr = [6, 8, 0, 1, 3, 2, 5]
(indices: 0 1 2 3 4 5 6)

nextGreater = [8, -1, 1, 3, 5, 5, -1]

[8, -1, 1, 3, 5, 5, -1]



nextGreater[arr.length]




```
Stack<Integer> s
for ( int i = arr.length-1 to 0 ) {
    while ( !s.isEmpty() && arr[s.peek()] <= arr[i] ) {
        s.pop();
    }
    if ( s.isEmpty() ) { nextGreater[i] = -1; }
```

```
else {
    nextgreater[i] = arr[s.peek()];
}

s.push(i);
}
```

Question 5   Stack Span problem

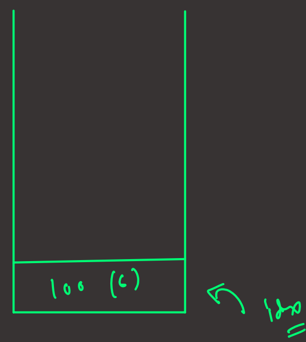
Stock price = [100, 80, 60, 70, 60, 85, 100]

0	1	2	3	4	5	6
100	80	60	70	60	85	100

find out : max no. of consecutive days for which price ≤ today's price.

Span o/p:

1	1	1	2	1	5	7



idx prevHigh [] =

0	1	2	3	4	5	6
-1	0	1	1	3	0	-1

pH =

-1	0	1	1	3	0	-1

Span

$$\boxed{\text{day } i = i - \text{prevHigh}[i]}$$

day 0 = 0 − (−1) = 1

day 1 = 1 − 0 = 1

day 2 = 2 − 1 = 1

day 3 = 3 − 1 = 2

day 4 = 4 − 3 = 1

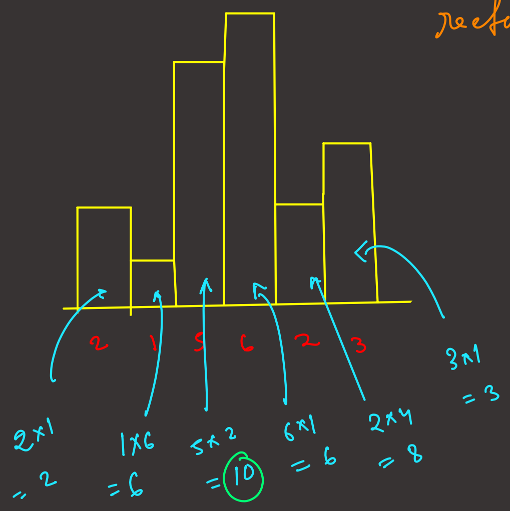
day 5 = 5 − 0 = 5

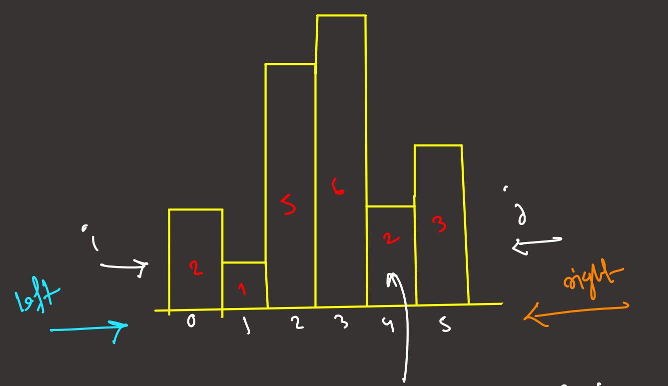
day 6 = 6 − (−1) = 7

Question 6

Max Area in a Histogram. height = [2, 1, 5, 6, 2, 3]

Given a array of integer heights, representing the histogram's bar height where the width of each bar is 1. Return the area of the largest rectangle in the histogram.





width = j - i - 1

(i) next smaller left = | -1 | -1 | 1 | 2 | 1 | 4 |

(j) next smaller right = | 1 | 6 | 4 | 4 | 6 | 6 |

n = arr. length

Stack

width[0] = 1-(-1) - 1
= 2-1 = 1

width[1] = 6 - (-1) - 1
= 6 + 1 - 1 = 6